

Teach Module 2: Java Syntax and Core Constructs

This README teaches Module 2 in original words and examples. It uses the course document only to identify the module topic list.

Module Topics

1. Primitive and reference types
2. Control flow statements
3. Packages and imports
4. Console input and output
5. Java naming and style conventions

1. Primitive and Reference Types

Java has two broad categories of data types: primitive types and reference types.

Primitive types store simple values directly.

```
int age = 25;
double price = 19.99;
char grade = 'A';
boolean isActive = true;
```

Common primitive types:

```
byte    // small whole number
short   // whole number
int      // most common whole number
long    // large whole number
float    // decimal, less precise
double  // decimal, more common
char     // single character
boolean // true or false
```

Example:

```
public class Main {
    public static void main(String[] args) {
        int quantity = 3;
        double unitPrice = 12.50;
        double total = quantity * unitPrice;

        System.out.println(total);
    }
}
```

Output:

37.5

Reference types store a reference to an object.

```
String name = "Asha";  
int[] scores = {90, 85, 88};
```

A `String` is not primitive. It is an object type.

Important difference:

```
int x = 10;  
String city = "New York";
```

`x` holds the actual number. `city` points to a `String` object.

2. Control Flow

Control flow means deciding which code runs and how many times it runs.

if / else

```
int score = 82;  
  
if (score >= 90) {  
    System.out.println("A");  
} else if (score >= 80) {  
    System.out.println("B");  
} else {  
    System.out.println("Needs improvement");  
}
```

switch

Use `switch` when checking one value against multiple fixed options.

```
int day = 2;  
  
switch (day) {  
    case 1:  
        System.out.println("Monday");  
        break;  
    case 2:  
        System.out.println("Tuesday");  
        break;  
    default:  
        System.out.println("Unknown day");  
}
```

for loop

Use a `for` loop when you know how many times to repeat.

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

while loop

Use a `while` loop when repetition depends on a condition.

```
int count = 1;  
  
while (count <= 5) {  
    System.out.println(count);  
    count++;  
}
```

3. Packages and Imports

A package is like a folder or namespace for Java classes.

```
package com.example.app;
```

Imports let you use classes from other packages.

```
import java.util.Scanner;  
import java.time.LocalDate;
```

Example:

```
import java.time.LocalDate;  
  
public class Main {  
    public static void main(String[] args) {  
        LocalDate today = LocalDate.now();  
        System.out.println(today);  
    }  
}
```

Java automatically imports `java.lang`, so you do not need to import `String`, `System`, `Math`, and similar core classes.

4. Console Input and Output

Output uses `System.out.println`.

```
System.out.println("Hello");
```

Input commonly uses `Scanner` .

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        System.out.println("Hello " + name + ", age " + age);
    }
}
```

Common `Scanner` methods:

```
nextLine() // reads a full line of text
next()     // reads one word
nextInt()  // reads an int
nextDouble() // reads a decimal
```

5. Naming and Style Conventions

Java style matters because clean code is easier to read.

Class names use `PascalCase` .

```
public class BankAccount {
}
```

Variable and method names use `camelCase` .

```
int accountBalance;
calculateTotal();
```

Constants use uppercase with underscores.

```
final double TAX_RATE = 0.08;
```

Good naming:

```
int studentAge;
double monthlySalary;
boolean isLoggedIn;
```

Weak naming:

```
int x;
double d;
boolean flag;
```

Unless the variable is tiny and obvious, use meaningful names.

Mini Practice

Write a Java program that asks the user for:

1. Their name
2. Their age
3. Their favorite programming language

Then print:

```
Hello Maya.
You are 24 years old.
Your favorite language is Java.
```

Starter:

```
import java.util.Scanner;

public class ProfileApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // your code here
    }
}
```

Practice Exercises

Beginner Exercises

1. Profile Printer: ask the user for their name, age, city, and favorite language. Print a formatted profile.
2. Simple Calculator: ask for two numbers and an operator: `+`, `-`, `*`, `/`. Print the result using `if/else` or `switch`.
3. Even or Odd Checker: ask for an integer and print whether it is even or odd.
4. Grade Calculator: ask for a score from `0` to `100`. Print `A`, `B`, `C`, `D`, or `F`.
5. Temperature Converter: convert Celsius to Fahrenheit and Fahrenheit to Celsius.

```
fahrenheit = (celsius * 9 / 5) + 32;
celsius = (fahrenheit - 32) * 5 / 9;
```

Control Flow Practice

6. Login Attempt Checker: hardcode a username and password. Ask the user to enter both. Print whether login succeeds.
7. Number Guessing Game: hardcode a secret number. Let the user guess until they get it right.
8. Multiplication Table: ask for a number and print its multiplication table from 1 to 10 .
9. Sum From 1 To N: ask for a number `n` . Use a loop to calculate the sum from 1 to `n` .
10. Menu-Based Program: show a menu and use `switch` to run the selected option.

1. Say Hello
2. Show Current Year
3. Exit

Types and Input Practice

11. Bill Splitter: ask for total bill amount, number of people, and tip percentage. Print how much each person pays.
12. Age Category Finder: ask for age and print `Child` , `Teenager` , `Adult` , or `Senior` .
13. BMI Calculator: ask for weight and height. Calculate BMI.

```
bmi = weight / (height * height);
```

14. Character Type Checker: ask for a character and check whether it is a vowel or consonant.
15. String Length Checker: ask for a word or sentence and print how many characters it has.

Packages, Imports, and Style Practice

16. Date Printer: use `java.time.LocalDate` to print today's date.
17. Random Number Generator: use `java.util.Random` to generate a random number from 1 to 100 .
18. Scanner Practice App: use `Scanner` to collect multiple values: name, salary, employee ID, and active status.
19. Constants Practice: create constants and use them in calculations.

```
final double TAX_RATE = 0.08;
final int DAYS_IN_WEEK = 7;
```

20. Naming Cleanup Exercise: rewrite poorly named variables into meaningful names.

```
int x = 25;
double y = 50000;
boolean z = true;
```

Best Mini Project for Module 2

Build a console banking menu.

```
Welcome to Simple Bank
```

1. Check Balance
2. Deposit
3. Withdraw
4. Exit

Requirements:

1. Use `Scanner` .
2. Use `double` `balance` .
3. Use a `while` loop to keep the menu running.
4. Use `switch` for options.
5. Prevent withdrawing more than the current balance.
6. Print clear messages.

This project combines primitives, variables, input/output, loops, conditions, and clean naming.

Lab: Java Syntax and Console Banking App

Goal

Build a small console-based banking program using Java basics.

Skills Practiced

1. Primitive types
2. `String`
3. `Scanner`
4. `if/else`
5. `switch`
6. `while` loop
7. Console input/output
8. Naming conventions

Lab Task

Create a Java file named:

```
SimpleBankApp.java
```

Build this menu:

```
Welcome to Simple Bank
```

1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit

The program should keep showing the menu until the user chooses `4` .


```

        break;

    case 4:
        isRunning = false;
        System.out.println("Thank you for using Simple Bank.");
        break;

    default:
        System.out.println("Invalid option. Please choose 1 to 4.");
    }
}

scanner.close();
}
}

```

Run Commands

```

javac SimpleBankApp.java
java SimpleBankApp

```

What to Test

Try these cases:

```

Check balance
Deposit 100
Withdraw 50
Withdraw more than balance
Deposit -20
Choose invalid menu option 9
Exit

```

Challenge Add-Ons

1. Add customer name input at the start.
2. Show the customer name with the balance.
3. Add a minimum balance rule.
4. Count how many transactions were completed.
5. Format money to two decimal places.

```

System.out.printf("Balance: $%.2f%n", balance);

```

Key Takeaway

Java programs are built from values, decisions, loops, organized classes, and readable syntax. Once these feel natural, object-oriented Java becomes much easier.